

Using neural networks for forecasting volatility of S&P 500 Index futures prices

Shaikh A. Hamid^{a,*}, Zahid Iqbal^b

^a*School of Business, Southern New Hampshire University, 2500 North River Road, Manchester, NH 03106, USA*

^b*School of Business, Texas Southern University, Houston, TX 77004, USA*

Received 3 January 2002; accepted 11 December 2002

Abstract

The ability to forecast the volatility of the markets is critical to analysts. Among the large array of approaches available for forecasting volatility, neural networks are gaining in popularity. We present a primer for using neural networks for financial forecasting. We compare volatility forecasts from neural networks with implied volatility from S&P 500 Index futures options using the Barone-Adesi and Whaley (BAW) American futures options pricing model. Forecasts from neural networks outperform implied volatility forecasts and are not found to be significantly different from realized volatility. Implied volatility forecasts are found to be significantly different from realized volatility in two of three forecast horizons.

© 2003 Elsevier Inc. All rights reserved.

Keywords: Neural networks; Volatility forecasting; Implied standard deviation; Realized standard deviation

1. Introduction

The ability to forecast volatility of the markets is important to analysts. This task is rendered complicated by the increasingly interconnected nature of financial markets of the world. This complexity has created the need for analytical tools that can explore interrelationships among a large number of market variables. This is where neural networks are gaining popularity.

Although neural networks have been around for almost 50 years, only since the late 1980s they have gained a major foothold in scientific and technical use. They have found applications in wide-ranging fields ranging from automated automobile inspection to sensor signal processing. The use of neural networks in economics and finance is a more recent phenomenon. The technology has been used in wide-ranging issues ranging from business cycle forecasting to valuation of real options. We have documented over 90 works in different areas of economics and finance (detailed references will be provided on request). The research works

involving the technology have been devoted predominantly to two areas: financial distress prediction (about 30% of the listed studies) and prediction of stock price/stock index (about 14%). Neural networks have yet to see other potential applications in financial economics. [Hawley et al. \(1990\)](#) mention a number of potential uses of neural networks on which academic research is yet to be seen—even after 12 years since then. For example, the technology can be applied to corporate finance (financial simulation, prediction, evaluation, etc.), IPO pricing, identifying arbitrage opportunities, security risk profiling, locating tax evaders, etc. Evaluation of uses in these areas is yet to be seen.

The general finding from the studies on the use of neural networks is that the technology has been found useful and has promising applications in many fields of economics, finance, and business. It has been found to outperform linear models in a variety of circumstances. However, the performance of the networks has not been consistent in all cases. They have been particularly effective in capturing complex relationships in which linear models fail to perform well.

Since the first draft of this article in 1995, neural networks have been used in volatility forecasting by some other authors. [Ormoneit and Neuneier \(1996\)](#) predict volatility of the German stock market using two types of networks. [Donaldson and Kamstra \(1997\)](#) have proposed the use of

* Corresponding author. Tel.: +1-603-644-3198; fax: +1-603-645-9737.

E-mail address: s.hamid@snhu.edu (S.A. Hamid).

neural network–GARCH model to capture volatility effects in stock returns. [González Miranda and Burgess \(1997\)](#) have used the networks to predict intraday volatilities for the Spanish stock market. [Schittenkopf et al. \(1998\)](#) predict the volatility of the Austrian stock market and find neural networks outperform ARCH models. [Schittenkopf et al. \(2000\)](#) use daily DAX data and find that volatility predictions from neural network are superior to GARCH models in that they have higher correlations with implied volatilities. [Meissner and Kawano \(2001\)](#) use a combined neural network–GARCH approach to capture the volatility smile of options on high-tech stocks.

The vast majority of the studies we have documented were published in nonfinance journals and nonfinance researchers authored many of them. This can lead one to wonder if finance/economics academics may still find neural networks to be esoteric “black box” enveloped in mystique. This article seeks to remove some of the mist. It explains in nontechnical terms what a neural network is and how a version of it works. A better understanding of the technology will hopefully generate needed research in the many unresolved issues relating to its applications in economics, finance, and business.

This article provides a primer on using neural networks for forecasting market variables in general, and in particular, forecasting volatility of the S&P 500 Index futures prices using over 10 years of daily data on a number of input variables. Volatility forecasts from neural networks are compared with implied volatility forecasts using [Barone-Adesi and Whaley \(1987\)](#) American futures options pricing model. Volatility forecasts from the two methods are then compared with realized volatility. Forecasts from neural networks outperform implied volatility forecasts. Volatility forecasts from neural networks are not found to be significantly different from realized volatility. Implied volatility forecasts are found to be significantly different from realized volatility in two of three forecast horizons.

The next section briefly explains neural network. Then we describe neural network architecture and operation of possibly the most commonly used type of network for forecasting—the back-propagation network. We then show how the networks can be used for forecasting volatility of the S&P 500 Index futures prices using data on a number of market variables. Finally, we present analysis of the results we obtain.

2. What is a neural network?

A neural network is a computational technique that benefits from techniques similar to ones employed in the human brain. It is designed to mimic the ability of the human brain to process data and information and comprehend patterns. It imitates the structure and operations of the three dimensional lattice of network among brain cells (nodes or neurons, and hence the term “neural”). The

technology is inspired by the architecture of the human brain, which uses many simple processing elements operating in parallel to obtain high computation rates. Similarly, the neural network is composed of many simple processing elements or neurons operating in parallel whose functions are determined by network structure, connection strengths, and the processing performed at computing elements or nodes.

The network’s strength is in its ability to comprehend and discern subtle patterns in a large number of variables at a time without being stifled by detail. It can also carry out multiple operations simultaneously. Not only can it identify patterns in a few variables, it also can detect correlations in hundreds of variables. It is this feature of the network that is particularly suitable in analyzing relationships among a large number of market variables. The networks can learn from experience. They can cope with “fuzzy” patterns—patterns that are difficult to reduce into precise rules. They can also be retrained and thus can adapt to changing market behavior. Even when a data set is noisy or has irrelevant inputs, the networks can learn important features of the data. Inputs that may appear irrelevant may in fact contain useful information. The promise of neural networks lies in their ability to learn patterns in a complex signal.

Time series models are one of the more widely used approaches for prediction purpose. Although useful, time series models pose a problem in the very beginning. Identification of the model (autoregressive versus moving average, or a combination of the two) that will fit a particular time series of data, and the order specification that will be appropriate is the difficult first step in using time series models. Neural networks do not depend on assumptions regarding the data but adapt to the data (see [Davies, 1995](#)). In addition, statistical models encounter difficulty when a data series is noisy. Neural networks are adept at handling such data.

A neural network can be described as a type of multiple regression in that it accepts inputs and processes them to predict some output. Like a multiple regression, it is a data modeling technique.

Neural networks have been found particularly suitable in complex pattern recognition compared to statistical multiple discriminant analysis (MDA) since the networks are not subject to restrictive assumptions of MDA models (see [Coats and Fant, 1993](#)).

3. Network architecture

Network architecture deals with arrangement of neurons into layers and the connection patterns within and between layers. The type of problem to be solved has a great deal to do with network architecture. The networks have been applied to solve problems in a wide variety of areas that include classification, filtering, pattern association, optimization, conceptualization, and prediction. For each prob-

lem-solving approach, more than one architecture may be used. Each architecture goes with numerous variations depending on parameters selected. Network parameters vary in factors as the following:

- The *number of layers* in the network through which input variables are processed, and the *number of neurons* or nodes per layer (neurons are the elements that process the inputs and learn about relationships between input and the output variables);
- *Connections between neurons* in each layer and the strength (weight) of each connection;
- *Transfer function*, through which the network seeks to relate the input data with the output data.

These factors are explained below.

3.1. Number of layers and neurons in each layer

It is helpful to visualize neurons as being grouped in layers. The number of layers will depend on the type and complexity of problem we explore. For prediction purpose, the most common form of architecture is probably the feed-forward multilayered network commonly termed “back-propagation.” A typical architecture incorporating back-propagation is shown in Fig. 1. Networks of this type have at least three layers. The first is the input layer that presents data into the network. This layer has as many neurons as there are input categories.

The next layer is called the “hidden layer.” A network may have more than one hidden layer depending on the complexity of the problem. The hidden layer(s), in conjunc-

tion with the input layer, creates an internal mapping of the input data. This process explores hidden patterns, correlations, and causal relationships in the data set to make generalizations. For financial forecasting, typically one hidden layer may be sufficient to map input data to output data.

No analytical formula can tell us how many hidden layers to use. That will need experimentation. How many neurons to use in the hidden layer? Too few neurons prevent the network from correctly mapping inputs into outputs. Too many neurons may cause the network to “memorize” trivial patterns that can impair its ability to “assimilate” important features or trends and make proper generalizations. [Mendelsohn \(1993\)](#) suggests hidden neurons between half the number of input variables and two times that number.

The final layer is the output layer. Each neuron in the output layer receives input from each neuron in the hidden layer immediately proceeding. The number of neurons in the output layer will equal the number of output categories we want.

In nearly all cases, a three- or four-layer network will do well; layers beyond four rarely perform better and increase the training time. [Qi \(1996\)](#) outlines a few optimal network guidelines that have been proposed.

3.2. Connections between neurons and their strengths

The neurons can be fully connected or partially connected. In the case of a fully connected network, all first layer neurons are connected to all hidden neurons in the next layer and all second layer neurons are connected to all third layer neurons. Finally, all neurons in the last hidden layer

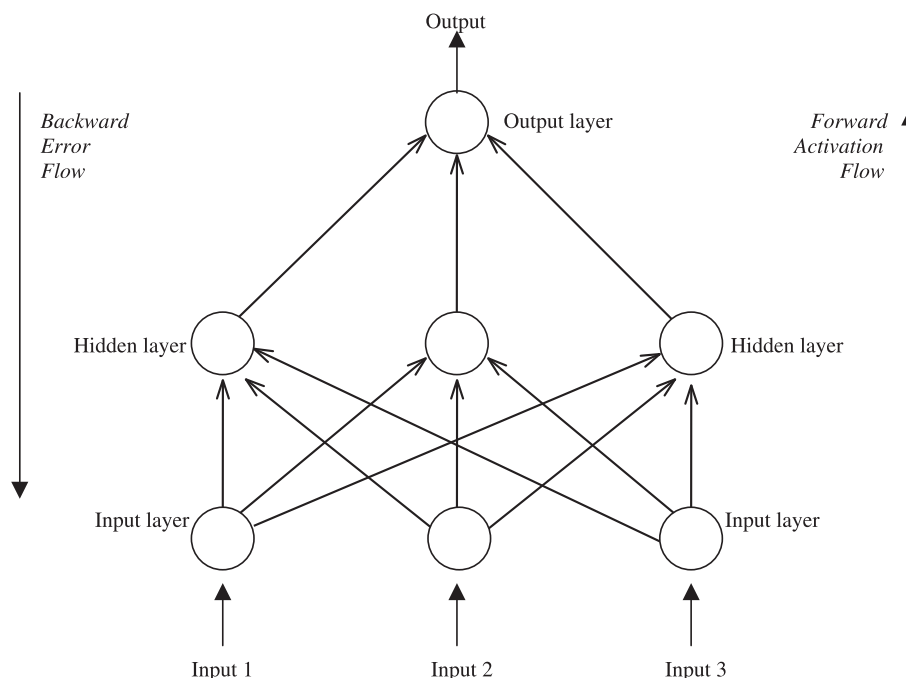


Fig. 1. A fully connected, three-layer network with feed-forward back-propagation configuration. The input and the hidden layers each have three neurons.

are connected to all neurons of the output layer. Fig. 1 also illustrates a fully connected network with three layers. The connections carry initial weights (connection strengths) that are altered during training. The network has to be fed with some random “seed values.” The seed values provide initial weights (hints) that the network learns to adjust in subsequent runs. The algorithm of the back-propagation network (called “generalized delta rule”) provides the “learning rule” or the method through which the network changes the connection weights during training, and thereby finds optimal values for the weights.

Apart from the architecture, neural networks set themselves apart by the way values of the weights are set for training purpose. In supervised training method, a network is trained by presenting it with a series of training cases (vectors) each with associated target output values. The weights are then adjusted based on the learning rule specified. In unsupervised training, a self-organizing network groups together similar input vectors without using training cases. It specifies characteristics of typical member of each group or the group to which each vector belongs. Weights are modified such that the most similar input values are assigned to the same output category.

3.3. Transfer function

Through this function a network seeks to make sense of the input data. The position and role of a transfer function are illustrated in Fig. 2. The figure also illustrates the basic functioning of an individual neuron. Each of the interconnected processing elements—or neurons—in a neural network sends and receives data to and from other processing elements. Input data ($I_1, I_2, \dots, I_n$) are multiplied by the weights ($W_1, W_2, \dots, W_n$) associated with the connection to the neuron. These products are then passed through a transfer function that converts the sum into a value in a specified interval between 1 and

–1 or 1 and 0. The output from this neuron is then multiplied by another, separate weight and fed into the next neuron. If a neuron is in the output layer, then the output from this neuron is not multiplied by a new weight. It becomes the output itself.

Without this transformation, the output value may be very large. A sigmoid transfer function dulls the effect of outliers. In addition to standard sigmoid, variations of sigmoid, Gaussian, hyperbolic tangent, and sine transfer functions are appropriately used in various problem-solving approaches.

3.4. Operation of the back-propagation network

We go back to Fig. 1 to understand the forecasting operation of a simple, fully connected, feed-forward back-propagation network. Feed-forward refers to network architecture, whereas back-propagation is a training method. To forecast, the network has to be trained using historical data—prices, volume, ratios, etc.

A single input category is fed into a single neuron in the input layer. Thus, there are as many neurons in the input layer as there are input categories. Each neuron multiplies the input data by some initial weight and passes on the value to every neuron in the hidden layer. Thus, each hidden layer neuron receives input from every input layer neuron in a fully connected network.

Each neuron in the hidden layer sums the values it receives and runs the summed value through a transfer function contained in it. The transfer function determines a weight with which the summed value is multiplied and then passed on to the single neuron in the output layer.

The neuron in the output layer receives values from all the neurons in the hidden layer. It sums the values, runs the result through its transfer function and multiplies the value by some weight to produce an output. The network compares this output with the desired output and determines the

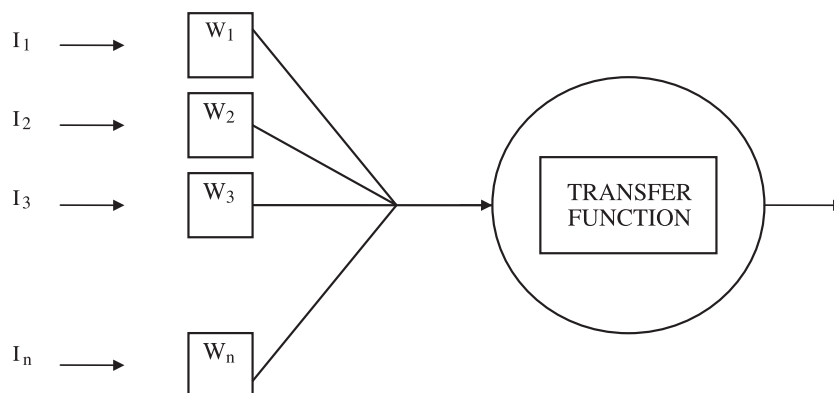


Fig. 2. Typical processing element or neuron of a neural network. Such individual interconnected processing elements are the brain cells of neural networks. A transfer function is specified in the neuron. The transfer function sums the weighted inputs $I_1W_1, \dots, I_nW_n$ and multiplies the summed value with a new weight determined by the type of the function. For financial forecasting, a nonlinear, continuously differentiable transfer function is needed. Adapted from Mendelsohn (1993) and Rumelhart et al. (1986).

difference between the two. This forms the error signal after the first run.

The error signal is fed back through the output layer and the hidden layer(s) to the first layer. As the error signal goes backward, each hidden neuron can determine how strongly it is connected to the output unit and the error at that neuron. A hidden neuron modifies its weight in proportion to the error times the input signal that reduces the error in the direction of most rapid reduction in error. The transfer function specified for each hidden neuron provides the rule for adjusting the weights based on the magnitude of the error in the output neuron. The extent of change in a given weight, as per the generalized delta rule, is the derivative of the transfer function with respect to its total input (see Rumelhart et al., 1986). The process of feed-forward and back propagation of values continues so that the error between the output generated and the output desired is gradually minimized over a series of data runs. In this way, the network trains itself to generate output closer and closer to desired output. This process of trial and error enables the network to recognize patterns, relationships, and correlations between the input variables and the output category. When the network can hardly minimize errors as more input data are fed, it reaches a steady state and the output can then be used for testing. Lippmann (1987) provides back-propagation training algorithm.

4. Limitations of neural networks

A major shortcoming is that the steps or process through which a network produces an output cannot be debugged or decomposed. Another shortcoming is that a network may “overfit” the data. To prevent overfitting, one solution is to use “fuzzy logic,” which instructs the network not to be emphatic when its conclusion is tentative. Another approach is to use “genetic algorithm,” which also uses trial and error and its mechanism is similar to how evolution works by mutation and natural selection (see Ridley, 1993). Yet, another solution is not to use too many data columns.

The technology also suffers from lack of optimal algorithm to ensure the global minimum because of multi-minima error surface (see Qi, 1996).

The lack of formal theory behind neural networks to help in model building implies users need a certain degree of sophistication in terms of selection of input variables and specifying appropriate network architecture. One approach may be to use in-sample model selection criteria like Akaike’s information criteria (AIC) and Bayesian information criteria (BIC). However, Qi and Zhang (2001) find that the commonly used in-sample model selection criteria are not able to identify the best neural network model for out-of-sample prediction. That implies that trial and error will continue to be an essential aspect of use of the networks. However, some networks, as the one we use, provide coefficients to reflect the relative contribution of input

variables to the prediction of desired output. Finding correlations between input and output variables can also be helpful. The combined use of these two tools can at least ease the input selection issue.

Like factor analysis, neural networks cannot be used to confirm ex post the identity of causal factors. In addition, ex ante identification of factors does not provide strong grounds to assert causality even when we have a good empirical fit.

Few statistical concepts have been applied in the development of neural networks. Nevertheless, the technology bears relationship with statistical models (see Qi, 1996).

5. Experimental design

5.1. Data

We want to predict the volatility of the S&P 500 Index futures prices. Our raw data series consists of closing settlement prices of 16 nearest futures contracts and 3 spot indexes. We take the futures contract series that will mature in the nearest maturity month. The maturity months are March, June, September, and December. Seven of the 16 futures contracts are on commodities (silver, platinum, palladium, heating oil, copper, gold, crude oil), 3 are on Treasury obligations (Treasury notes, bonds, and bills), and 6 are on foreign currencies (Swiss frank, yen, mark, Canadian dollar, British pound, euro dollar). The three spot indexes are DJIA, NYSE Composite Index, and S&P 500 Index. We also use 1-day lagged S&P 500 futures prices as an additional explanatory variable for a total of 20 variables. We select these variables because of availability of 10 years of daily data on them—from February 1, 1984, to January 31, 1994—2531 observations per variable. The data set was obtained from Knight–Ridder Financial Publishing. Since neural networks need to be trained with a large data set, it fits well with our needs. From the raw data series, we calculate 20-day rolling historical standard deviations (HSDs). We calculate HSDs from daily percentage price changes of the 20 variables calculated as natural log relatives of the price or index series. The percentage change for Day 2 based on prices P_1 and P_2 will be given by: $\ln(P_2/P_1)$. We use 500 HSD observations to train the network and the rest for forecasting.

We obtain 90 forecasts; 30 of the forecasts are over 55 days following the forecast dates (55-day forecasts) and equal numbers of 35- and 15-day forecasts. These forecast dates and horizons correspond to implied volatility forecasts we obtain using the Barone-Adesi and Whaley (1987) American futures options pricing model.

5.2. Implied standard deviations from Barone-Adesi and Whaley futures options pricing model

The Barone-Adesi and Whaley (BAW) model is for pricing American options on futures that has no analytic

solution. We devise an algorithm for extracting implied standard deviations (ISDs) from the model. We execute the algorithm on a connection machine to extract the implied volatilities from the BAW model. (Connection machines are parallel processors that have speed beyond the reach of present-day PCs.) The implied volatilities are obtained from call options on S&P 500 Index futures contracts. We use just-out-of-the-money options on forecast dates to extract the implied volatilities. These are options for which futures price minus exercise price is nearest to 1 but negative.

The futures and options contract series run from January 1986 to June 1993—four contract series per year. For each futures contract series, we extract forecasts at three different points in the life of the series. These three forecasts serve as forecasts for the three horizons: 55, 35, and 15 trading days to maturity of a futures contract. The forecast dates are 55, 35, and 15 days before the maturity of nearest futures contracts and corresponding options contracts. This enables us to compare forecasts from neural networks with implied volatility—a type of forecast that is highly regarded. We compare forecasts from neural network and from BAW model with volatility realized over each of the three forecast horizons. Since the realized volatilities are obtained on the basis of trading days as opposed to calendar days, to be consistent we modify the BAW model to get ISDs based on trading days.

5.3. Volatility realizations

We derive 55-day realized standard deviation (RSD) from daily log relatives of the present value of S&P 500

Index futures settlement prices from 55 days before maturity until the day of maturity. For the 35- and 15-day forecast horizons, the daily returns are based on daily log relatives of the present value of the index futures settlement prices, respectively, from 35 and 15 days before maturity until the day of maturity. RSD on day t is calculated as follows:

$$RSD_t = \left[\sum_{j=t}^{t+n} (R_j - \bar{R})^2 / n \right]^{1/2} \quad (1)$$

where $R_j = \ln [F_j / F_{(j-1)}]$, $\bar{R} = R_j / n$, and F_j is the present value of futures price on date j , $n = 55, 35$, and 15 , respectively, for RSDs over the three horizons.

As Fig. 3 shows, each 55-day forecast from a futures contract series will be nonoverlapping with the 55-day forecast from the previous or subsequent contract series. This will ensure that the forecast error of each period will be uncorrelated with the errors of other periods. The error of the 35-day forecast obtained from a series will be uncorrelated with the errors of other 35-day forecasts. Similar is the case of errors of 15-day forecasts.

Then, we compare the two sets of 55-day forecasts with 55-day volatility realizations. Similarly, we separately evaluate the accuracy of the 35-day forecasts and the 15-day forecasts by comparing with corresponding volatility realizations.

5.4. Forecast accuracy

To measure forecast accuracy, we calculate mean of absolute errors (MAE) and root mean of squared errors

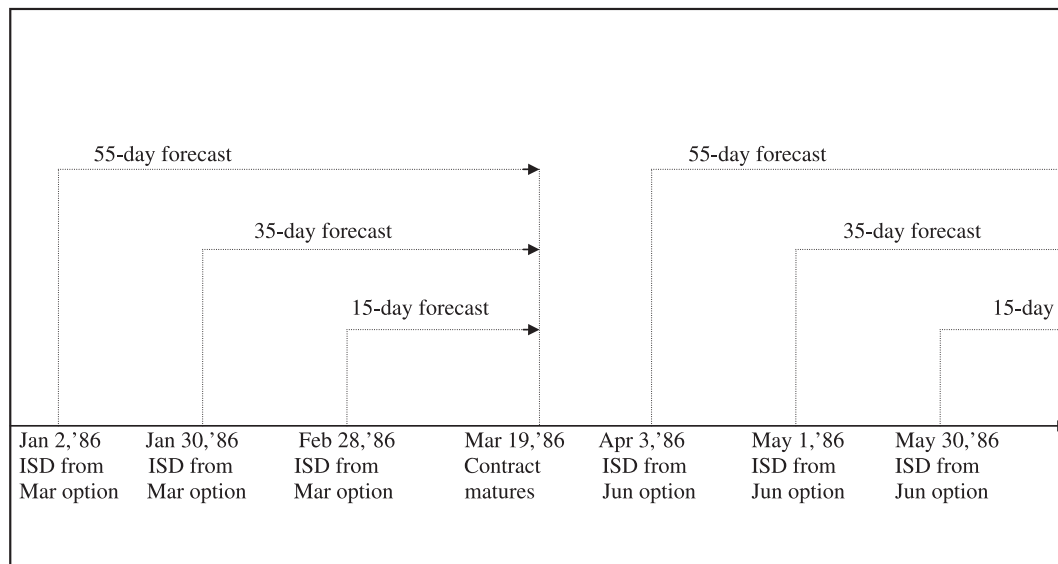


Fig. 3. Nonoverlapping forecast horizons. The errors of 55-day forecast made on January 2, 1986, will be uncorrelated with the errors of the next 55-day forecast (made on April 3, 1986). Similarly, the errors of 35-day forecast made on January 30, 1986, will be uncorrelated with the errors of the next 35-day forecast (made on May 1, 1986). Similar is the case with 15-day forecasts. Thus, each forecast of a particular horizon will be uncorrelated with previous or subsequent forecasts.

(RMSE) of volatility forecasts compared to realized volatility for the three forecast horizons, as follows:

$$\text{MAE} = (1/30) \sum_{i=1}^{30} [\text{Abs}(Y_{it} - \hat{Y}_{it})] \quad (2)$$

$$\text{RMSE} = \left[(1/30) \sum_{i=1}^{30} (Y_{it} - \hat{Y}_{it})^2 \right]^{1/2} \quad (3)$$

where $\hat{Y}_{it}$ is the forecasted standard deviation, t is the forecast horizon (15, 35, and 55 days), and Y_{it} is the RSD.

We also separately test for the differences in the means of each type of forecast with respect to the means of realized volatility for each of the three forecast horizons using standard test statistics. Since the standard t tests assume normal distribution of the data and our data may not be normally distributed, we also perform a nonparametric test—the Mann–Whitney test. This test performs a two-sample rank test for the difference between two population medians.

6. Volatility forecasts

6.1. Steps in forecasting volatility

We use the following steps to forecast volatility from a back-propagation neural network using the 20-day rolling HSD series of the input variables.

- (a) Selection of input variables
- (b) Preprocessing the input data
- (c) Specifying a neural network
- (d) Training the network and forecasting

These four steps are explained below.

6.1.1. Selection of input variables

This step identifies the variables that contribute the most to forecasting the target variable. Too many variables can unnecessarily overload the system. If we omit important variables, then its effect on the performance of the neural network can be significant. Our perspective on the markets will affect the choice of input data.

The larger the number of input variables, the longer will be the training period required and greater the possibility that the data will overfit the model determined by the network. As an upper limit to the number of input variables, Kean (1993) suggests around 10% of the number of data observations. Thus, if there are 300 days of observations to train a network, Kean recommends 30 variables.

The selection of input variables will depend on the knowledge of what affects the target variable and the use of statistical tools to find correlation between the target and the other variables. It can be a lengthy process of trial and

error. Multiple regression analysis can help to identify statistically significant variables that can be used as input variables; principal component analysis and stepwise regression analysis would also be helpful.

To simplify this step, we initially included 20 explanatory variables mentioned earlier. We find the correlations of the daily price changes of 18 of the 20 variables with the daily price changes of S&P 500 Index futures contracts. (We do not calculate the correlation of the S&P 500 Index futures prices with its lag or with S&P 500 Index since the correlations will be very high.) From the 20 variables, we select 13 explanatory variables (NYSE, S&P 500 Index, 1-day lagged S&P 500 Index futures, futures on all seven commodities, Treasury bonds and bills, and yen).

In general, a variable is selected if it meets one of the two conditions:

- correlation with futures prices is greater than 5% and less than -5% ; or,
- high relative contribution to forecasting (relative contribution coefficient greater than .07 is the criteria used; this coefficient is provided by the network we use and is explained in McCormick, 1992).

This provides us with 16 explanatory variables. Under the apprehension that this number may be on the high side—in which case the network may “overfit” the data—we dropped three more variables. We dropped the Canadian dollar (relative contribution coefficient of .33) because of its very low correlation (.005) with S&P 500 Index futures prices. We dropped euro dollar in spite of its modest correlation with S&P 500 Index futures prices (.15) and modest relative contribution coefficient (.23). We also dropped DJIA (relative contribution coefficient of .078 and correlation of .94) under the assumption that its effect will be captured by the included variable—S&P 500 Index. This implies there is scope for obtaining better forecasts from the networks by using a different number of input variables than we used.

6.1.2. Preprocessing the input data

Neural networks need properly transformed data to be able to process them and generate sound forecasts. Transformation, normalization, and data smoothing are three common ways of preprocessing data. Through transformation, we can coalesce a few input variables to form a single input category. Methods include taking differences between inputs or ratios of inputs. Reducing the inputs may help the network learn better. However, it is not necessary to transform data before feeding into a network.

Normalization makes the statistical distribution of each input and output data roughly uniform. The values are scaled to match the range that the input neurons use. Data normalization methods, which include simple linear scal-

ing and statistical measures of central tendency and variance, remove outliers and spread out the distribution of the data.

Data smoothing filters out noise in the data. Smoothing techniques suggested are simple and exponential moving averages, and polynomial regression.

Some of the networks available in the market have limited built-in preprocessing capabilities like scaling and randomly rearranging data to remove serial dependence. However, these networks cannot transform or smooth data. If we need to transform or smooth data, we have to do that before feeding the data into the system.

Preprocessing has two parts: (i) arranging the data in the form that the neural network can read and (ii) scaling the data so that the maximum and minimum of each variable falls in a range of 1 and -1 (or 0 and 1 depending on the type of transfer function specified), respectively, and the other values are scaled accordingly.

6.1.3. Specifying a neural network

Since a typical back-propagation network should have at least three layers, we specify a three-layered network. Appropriate specification of number of layers is an art. It needs experimentation.

The number of neurons in the first layer—13—is equal to the number of explanatory variables. We specified two times that many neurons in the second layer. Lesser number of neurons could also have been specified. However, since we use three layers rather than four, we wanted to be rigorous in terms of number of neurons in the hidden layer. More layers and/or neurons would have increased the training time for each of the desired 90 forecasts. The results would also change.

In the network specification stage, we can adjust a number of default parameters or values that influence the behavior of the training process. These deal with the learning, forgetting, and error tolerance rates of the network, the overlearning threshold, the maximum number of runs, stop value for terminating training, and randomizing weights with some specified dispersion. In the absence of objective guidelines, we set most of these parameters to default values of the network.

6.1.4. Training the network and forecasting

A neural network learns from past experience and so has to be trained with a sufficient number of training cases. (This contrasts with expert systems, which have to be fed with rules.) The use of too few training cases will cause the network to map an inadequate model for the output. One rule of thumb suggests four times as many test cases as weights. That means a network with 100 weights needs to be fed with at least 400 training cases. The time taken to train the network will depend on the number of layers, the neurons per layer, the number of iterations on each day's data set and the speed of the computer. Much of the art of using neural networks comes into play in the

training phase. Among the decisions involved at this stage are:

- the number of layers and neurons per layer: these can be varied depending on the performance of the network during the training phase;
- type of transfer function: the standard sigmoid is the most commonly used for market forecasting;
- number of times each day's data set will run through the system;
- the learning rate: extent of correction applied to the connection weights at each step of training; this is not a crucial decision, it can be set to default value.

We train the network using the 13 explanatory variables mentioned earlier. We use 500 days of data for training purpose. Each set of observations is run 500 times through the network. After each run, the network compares the forecasted volatility of futures prices with the desired volatility. It calculates and feeds the error backward. The neurons reset their weights each time the errors are fed back.

The desired volatility on a particular day for the 55-day forecast horizon is the volatility realized in the subsequent 55 days. The desired volatility on a particular day for the 35-day forecast horizon is the volatility realized in the subsequent 35 days. For the 15-day horizon, the desired volatility on a particular day is the 15-day volatility realization.

After each volatility forecast, we change the training observations. In each case, we use 500 twenty-day rolling HSD observations for training. These observations are for the 500 trading days before a forecast date. In all, we get 30 forecasts for each of 55, 35, and 15 trading days—a total of 90 forecasts.

7. Analysis of results

Table 1 shows the RSD, volatility forecasts from neural networks (neural), and ISDs using BAW American futures options pricing model. The realized and forecasted volatilities are shown for the three forecast horizons of this study: 55, 35, and 15 days to the maturity of nearest futures and corresponding options contracts. For each forecast class, we have 30 forecasts. The bottom of the table shows the means of RSD, neural, and ISD. No significant differences in the means in the case of all three forecast classes t tests for differences in means of RSD versus neural, show no significant differences, whereas t tests for differences in means of RSDs versus ISDs show significant difference in the means of 15- and 35-day forecasts ($P=.00$ and $.01$, respectively, in one-tailed test) and no significant difference in the case of 55-day forecasts ($P=.44$ in one-tailed test). That means, whereas ISDs have provided good forecasts over the 55-day horizon, neural forecasts have been good in the case of all three horizons. Results of Mann–Whitney nonparametric tests

Table 1

Means of RSDs and forecasts from neural networks and ISD for 15-, 35-, and 55-day forecast horizons

	15-day horizon			35-day horizon			55-day horizon		
	RSD	Neural	ISD	RSD	Neural	ISD	RSD	Neural	ISD
Mean	.008642	.009533	.010264	.009207	.010033	.011625	.011340	.010700	.011625
S.D.	.003453	.003461	.003098	.003162	.005840	.004942	.009382	.007349	.004942
<i>t</i> statistics		−1.00	−1.92		−0.68	−2.26		0.29	−0.15
<i>P</i> (two-tailed)		(.20)	(.00)		(.49)	(.03)		(.77)	(.88)
<i>P</i> (one-tailed)		(.10)	(.00)		(.25)	(.01)		(.38)	(.44)
MAE		.002447	.002598		.002535	.003079		.001923	.002379
RMSE		.003237	.003114		.003996	.004114		.003432	.005113
<i>Mann–Whitney nonparametric test results</i>									
Median	.007760	.009000	.009740	.008500	.009000	.010600	.009090	.009000	.010600
<i>W</i> statistics		823	738		912	727		949	812
<i>P</i> (two-tailed)		.18	.01		.97	.01		.62	.13

The *t* statistics for tests of differences in the means of forecasts with respect to RSDs are shown with *P* values. MAEs and RMSEs of forecasts with respect to RSDs are also shown. Statistically significant results are in bold.

yield similar conclusions: in two-tailed tests, neural forecasts do not show significant differences from realized volatilities in the case of all three forecast classes; ISDs are significantly different from volatility realizations in the case of 15- and 35-day forecast classes ($P=.01$ in both cases) and not so significantly different in the case of 55-day forecast class ($P=.13$).

It clearly implies that for the data in our sample, volatility forecasts from neural networks provided superior results compared to implied volatilities.

Table 1 also shows the MAEs and RMSEs of the two types of forecasts. Except for the RMSE of ISDs for 15-day forecast class, on both measures, and for all three forecast classes, neural forecasts outperformed ISDs.

8. Conclusion

The use of neural networks in finance is a promising field of research specially given the ready availability of large mass of data sets and the reported ability of neural networks to detect and assimilate relationships among a large number of variables. However, the realization of the potentials of neural networks in forecasting the market involves more of an art than science. There are many ways and combinations in which a neural network can be specified and many types of data can be simultaneously used. Much of that aspect is yet unexplored. The principles that can guide us in effectively using the networks in financial applications remain a fertile area of research. In addition, the application of the technology is yet to be evaluated in some promising areas of economics, finance, and business.

9. Direction for future research

In our forecasting, we used 13 input variables. We also used 11 input variables; the results did not improve. Could it have improved with more than 13 variables? Could the

results have improved with a different number of neurons in the hidden layer, or with two rather than one hidden layer? Guidelines available at present are rather sketchy at best. A great deal of trial-and-error experimentation is called for. Programming skills are not essential in using a large number of commercial neural softwares that are available. This facilitates trial-and-error experimentation. A useful site listing many such softwares with links to description on them is: www.emsl.pnl.gov:2080/proj/neuron/neural/systems/software.html.

This article is intended as a primer on how to use neural networks for market forecasting. It makes a rather straightforward exposition of forecasting volatility of S&P 500 Index futures prices. However, the field is advancing. Extension of this research may proceed in the following directions: use simulated and real data to validate the robustness of results following Zhang (2001); specify both linear and nonlinear neural networks to compare the in-sample fitting and out-of-sample forecasts; use AIC and BIC criteria to add to the results following Qi and Zhang (2001); use a recursive model similar to Qi (2001) to capture possible structural changes in the economy that may have an impact on forecasts.

Acknowledgements

We gratefully acknowledge insightful suggestions and ideas provided by two anonymous reviewers.

References

- Barone-Adesi G, Whaley RE. Efficient analytic approximation of American option values. *J Finance* 1987;42:301–20.
- Coats PK, Fant LF. Recognizing financial distress patterns using a neural network tool. *Financ Manage* 1993;142–55.
- Davies PC. Neural network techniques for financial time series analysis. In: Lederman J, Klein RA, editors. *Virtual trading*. Chicago, IL: Probus Publishing Co.; 1995. p. 81–7.

- Donaldson RG, Kamstra M. An artificial neural network—GARCH model for international stock return volatility. *J Empir Evid* 1997;4:17–46.
- González Miranda F, Burgess N. Modeling market volatilities: the neural network perspective. *Eur J Financ* 1997;3:137–57.
- Hawley DD, Johnson JD, Raina D. Artificial neural systems: a new tool for financial decision-making. *Financ Anal J* 1990;(November/December): 63–72.
- Kean J. Neural nets and stocks: training a predictive system. *PC* 1993; A1(September–October):45–7.
- Lippmann RP. An introduction to computing with neural nets. *IEEE ASSP Mag* 1987;(April):4–22.
- McCormick DL. N-TRAIN: neural network development system. Users manual 1992; 1(2). Selden (NY): Scientific Consultant Services; 1992.
- Meissner G, Kawano N. Capturing the volatility smile of options on high-tech stocks—a combined GARCH–neural network approach. *J Econ Finance* 2001;25(3):276–93 [Fall].
- Mendelsohn L. Preprocessing data for neural networks. *Tech Anal Stocks Commod* 1993;52–8.
- Ormoneit D, Neuneier R. Experiments in predicting the German Stock Index DAX with density estimating neural networks. *Proc 1996 Conf Comput Intell Financ Eng (CIFER)* 1996;66–71.
- Qi M. Financial application of artificial neural networks. In: Maddala GS, Rao CR, editors. *Handbook of statistic: statistical methods in finance*, vol. 14. Amsterdam: Elsevier; 1996. p. 529–52.
- Qi M. Predicting US recessions with leading indicators via neural network models. *Int J Forecast* 2001;17:383–401.
- Qi M, Zhang P. An investigation of model selection criteria for neural network time series forecasting. *Eur J Oper Res* 2001;132:666–80.
- Ridley M. Frontiers of finance. *Economist* 1993;(October):9–15.
- Rumelhart DE, Hinton GE, Williams RJ. Learning internal representations by error propagation. In: Rumelhart DE, McClelland JL, editors. *Parallel distributed processing: explorations in the microstructures of cognition*. Cambridge (MA): MIT Press; 1986. p. 318–62.
- Schittenkopf C, Dorffner G, Dockner EJ. Volatility predictions with mixture density networks. In: Niklasson L, Bodén M, Ziemka T, editors. *Proceedings of the 8th International Conference on Artificial Neural Networks. ICANN 98*. Berlin: Springer; 1998. p. 929–34.
- Schittenkopf C, Dorffner G, Dockner EJ. Forecasting time-dependent conditional densities: a semi-nonparametric neural network approach. *J Forecast* 2000;19(4):355–74 [July].
- Zhang GP. An investigation of neural network for linear time-series forecasting. *Eur Comput Oper Res* 2001;28(12):1183–202 [October].